Университет ИТМО
Мегафакультет трансляционных информационных технологий
Факультет информационных технологий и программирования

Группа: M3206

К работе допущен:

Студент: Абубакиров Аскар

Работа выполнена:

Преподаватель: Гумбатов Владислав Юрьевич

Отчет принят:

Рабочий протокол и отчет по лабораторной работе № 2

Идентификация процессов

---

1. Напишите скрипт, выводящий PID всех процессов, состояние которых относится к множеству {R, S, D, Z, T}.

```
#!/bin/bash

ps a | awk '$3 ~ /^[RSDZT]/ { print $1 }'
```

```
[root@localhost lab2]# ./task1.sh
PID
1448
1449
1707
2403
2409
2923
3574
3575
3576
```

2. Напишите скрипт, выводящий PID процесса с наибольшим временем непрерывной работы (наиболее «долго работающего»).

```
#!/bin/bash

ps -Ao pid=,etimes= --sort etimes | tail -n 1
```

```
[root@localhost lab2]# ./task2.sh
30 51513
```

3. Напишите скрипт, выводящий PID последнего запущенного процесса.

```
#!/bin/bash

ps -Ao pid=,etimes= --sort etimes | head -n 1
```

```
[root@localhost lab2]# ./task3.sh
   3858       0
```

4. Создайте процесс командой *sleep 10000 &*.
   1. Определите PID созданного процесса и перейдите в каталог */proc/ < PID > /* .
   2. Выведите последние 5 строк стека вызовов процесса.
   3. Выведите поля Name, PPid, Kthread, а также количество потоков процесса.
   4. Объясните, почему значения Tgid и Pid совпадают (для данного случая).

```
#!/bin/bash

# strace -i ./troubleshooting_amd64

strace -e trace=network -o troubleshooting_network.log ./troubleshooting_amd64

strace -e trace=file -o trobuleshooting_file.log ./troubleshooting_amd64
```

```
#!/bin/bash

strace -i ./troubleshooting_amd64
```

   Pid - индентификатор потока, Tgid - индентификатор группы потоков, т е процесса. У процесса sleep 10000 обычно один поток, поэтому совпадает.

5. Напишите скрипт, завершающий все процессы, которые работают менее (n) секунд, где (n) задаётся параметром. Запишите в файл *killed.log* PID завершённых процессов (по одному PID в строке).

```
#!/bin/bash

ps -Ao pid=,etimes= | while read pid time; do
        if [ "$time" -lt "$1" ] && [ "$pid" -ne "$$" ]; then
                kill "$pid" 2>/dev/null && echo "$pid" >> killed.log
        fi
done
```

```
[root@localhost lab2]# ./task5.sh 1000
./task5.sh: line 7:  4090 Done                    ps -Ao pid=,etimes=
      4091 Terminated              | while read pid time; do
    if [ "$time" -lt "$1" ] && [ "$pid" -ne "$$" ]; then
        kill "$pid" 2> /dev/null && echo "$pid" >> killed.log;
    fi;
done
[root@localhost lab2]# cat killed.log
4007
4040
```

6. Выведите PID процессов и суммарное потребление оперативной памяти; отсортируйте вывод по потреблению памяти.

```
#!/bin/bash

ps -Ao pid=,rss= --sort=rss

ps -Ao rss | awk '{sum+=$1} END {print "Total ram: ", sum}'
```

```
[root@localhost lab2]# ./task6.sh
      2     0
      3     0
      4     0
      6     0
      7     0
      8     0
      9     0
     10     0
     11     0
     12     0
     13     0
     15     0
     16     0
     17     0
     18     0
     19     0
     20     0
     21     0
     22     0
     23     0
     24     0
     25     0
     26     0
     27     0
     28     0
     29     0
     30     0
     53     0
    144     0
    145     0
    146     0
    147     0
    148     0
    149     0
    150     0
    454     0
    455     0
    456     0
    457     0
    458     0
    459     0
    460     0
    461     0
    462     0
    463     0
    465     0
    466     0
    467     0
    471     0
```

```
    776     0
    777     0
    779     0
    780     0
    781     0
    803     0
    810     0
    811     0
    813     0
    841     0
    852     0
    853     0
   1039     0
   1054     0
   1407     0
   2124     0
   2138     0
   2629     0
   2692     0
   2708     0
   2713     0
    844     8
    872     8
    903     8
    838    12
    851   220
    806   324
    868   548
    835   604
    836   628
   1401  1060
   1047  1772
    847  1800
    843  1880
    900  2044
    839  2132
   1783  2408
   1413  3108
   2742  3152
    869  3196
   2743  3644
    667  3664
    876  3748
    888  4136
   2706  4784
   2714  5296
   2707  5596
      1  6352
Total ram:  64248
[root@localhost lab2]#
```

7. Имеется бинарный файл `troubleshooting`, завершающийся с кодом `1` без вывода диагностических сообщений. Для выполнения задания используйте `strace`. Учтите, что программа завершается через 20 секунд.

   1. Задание считается выполненным, если программа завершится с кодом `0` и выведет сообщение `You passed`.

```
#!/bin/bash

strace -i ./troubleshooting_amd64
```

```
[00000000000408c6e] openat(AT_FDCWD, "/root/requiredDir/requiredFile", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
[00000000004895eb] exit_group(1)        = ?
[????????????????] +++ exited with 1 +++
[root@localhost lab2]#
```

   Заметим, что программа пытается открыть файл `/root/requiredDir/requiredFile`, но его не существует (`No such file or directory`), поэтому программа завершается с кодом **1**. Создадим данный файл.

   2. С помощью `strace` выведите сетевые и файловые операции в файлы `troubleshooting_network.log` и `troubleshooting_file.log` соответственно (используйте параметр `-e trace=...`).

```
#!/bin/bash

# strace -i ./troubleshooting_amd64

strace -e trace=network -o troubleshooting_network.log ./troubleshooting_amd64

strace -e trace=file -o trobuleshooting_file.log ./troubleshooting_amd64
```

```
[root@localhost lab2]# cat troubleshooting_network.log
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2487, si_uid=0} ---
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, 16) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(58754), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, [112->16]) = 0
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 8
setsockopt(8, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(8, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, 16) = 0
getsockname(8, {sa_family=AF_INET, sin_port=htons(47620), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(8, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, [112->16]) = 0
socket(AF_INET6, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_IPV6, IPV6_V6ONLY, [0], 4) = 0
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET6, sin6_port=htons(53), inet_pton(AF_INET6, "2a00:1450:4026:800::200e", &sin6_addr), sin6_flowinfo=htonl(0), sin6_scope_id=0}, 28)
= -1 ENETUNREACH (Network is unreachable)
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("216.58.210.142")}, 16) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(58054), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("216.58.210.142")}, [112->16]) = 0
socket(AF_INET, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
connect(5, {sa_family=AF_INET, sin_port=htons(443), sin_addr=inet_addr("216.58.210.142")}, 16) = -1 EINPROGRESS (Operation now in progress)
getsockopt(5, SOL_SOCKET, SO_ERROR, [0], [4]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(443), sin_addr=inet_addr("216.58.210.142")}, [112->16]) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(34268), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
setsockopt(5, SOL_TCP, TCP_NODELAY, [1], 4) = 0
setsockopt(5, SOL_SOCKET, SO_KEEPALIVE, [1], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPIDLE, [30], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPINTVL, [15], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPCNT, [9], 4) = 0
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2487, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2487, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2487, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2487, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2487, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2487, si_uid=0} ---
+++ exited with 1 +++
```

```
[root@localhost lab2]# cat troubleshooting_file.log
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, 16) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(53954), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, [112->16]) = 0
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 8
setsockopt(8, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(8, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, 16) = 0
getsockname(8, {sa_family=AF_INET, sin_port=htons(54640), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(8, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("192.168.64.1")}, [112->16]) = 0
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("142.251.38.78")}, 16) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(52657), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr("142.251.38.78")}, [112->16]) = 0
socket(AF_INET6, SOCK_DGRAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
setsockopt(5, SOL_IPV6, IPV6_V6ONLY, [0], 4) = 0
setsockopt(5, SOL_SOCKET, SO_BROADCAST, [1], 4) = 0
connect(5, {sa_family=AF_INET6, sin6_port=htons(53), inet_pton(AF_INET6, "2a00:1450:4026:800::200e", &sin6_addr), sin6_flowinfo=htonl(0), sin6_scope_id=0}, 28)
= -1 ENETUNREACH (Network is unreachable)
socket(AF_INET, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, IPPROTO_IP) = 5
connect(5, {sa_family=AF_INET, sin_port=htons(443), sin_addr=inet_addr("142.251.38.78")}, 16) = -1 EINPROGRESS (Operation now in progress)
getsockopt(5, SOL_SOCKET, SO_ERROR, [0], [4]) = 0
getpeername(5, {sa_family=AF_INET, sin_port=htons(443), sin_addr=inet_addr("142.251.38.78")}, [112->16]) = 0
getsockname(5, {sa_family=AF_INET, sin_port=htons(58570), sin_addr=inet_addr("192.168.64.2")}, [112->16]) = 0
setsockopt(5, SOL_TCP, TCP_NODELAY, [1], 4) = 0
setsockopt(5, SOL_SOCKET, SO_KEEPALIVE, [1], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPIDLE, [30], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPINTVL, [15], 4) = 0
setsockopt(5, SOL_TCP, TCP_KEEPCNT, [9], 4) = 0
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2265, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2265, si_uid=0} ---
--- SIGUSR2 {si_signo=SIGUSR2, si_code=SI_USER, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
--- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=2265, si_uid=0} ---
+++ exited with 1 +++
```

В логах *strace* присутствуют строки с сигналом **SIGUSR2**, что означает получение процессом пользовательского сигнала.

3. По результатам анализа определите, какой из пользовательских сигналов — `SIGUSR1` или `SIGUSR2` — обрабатывается программой. Запустите программу в одном терминале, затем в другом отправьте процессу противоположный сигнал и проанализируйте результат. Например, если программой обрабатывается `SIGUSR1`, тогда противоположным сигналом будет `SIGUSR2`. Пункт считается выполненным, если программа выведет сообщение `You passed 7.3`.

```
[root@localhost lab2]# ./troubleshooting_amd64 &
[1] 2609
[root@localhost lab2]# kill -SIGUSR1 2609
[root@localhost lab2]# You passed 7.3
You passed
```

8. Запустите бинарный файл `top_troubleshooting`.
   1. С помощью `top` определите, какая проблема возникает.
   2. Опишите, что происходит с файлом через некоторое время.

```
[root@localhost lab2]# ./top_troubleshooting_amd64
[13534.324515] top_troubleshoo[2620]: segfault at 0 ip 0000000000401883 sp 00007ffdec38df70 error 6 in top_troubleshooting_amd64[401000+7e000]
[13534.324956] Code: 1f 84 00 00 00 00 00 e9 4b ff ff ff f3 0f 1e fa 55 48 89 e5 48 83 ec 10 bf a0 0f 00 00 e8 45 80 00 00 48 89 45 f8 48 8b 45 f8 <c7> 00 05 00
 00 00 90 eb e5 0f 1f 40 00 f3 0f 1e fa 55 48 8d 05 d4
[13534.365479] Core dump to |/usr/lib/systemd/systemd-coredump 2620 0 0 11 1773252097 0 localhost.localdomain top_troubleshoo pipe failed
Segmentation fault
[root@localhost lab2]#
```

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2624 root      20   0 2459896   2.3g    604 R  98.4  63.6   0:15.46 top_troubleshoo
 2625 root      20   0  274288   4444   3772 R   1.3   0.1   0:00.22 top
```

```
[14173.571968] Node 0 DMA32: 2*4kB (M) 2*8kB (ME) 2*16kB (UE) 3*32kB (UME) 2*64kB (UE) 2*128kB (UE) 2*256kB (UE) 2*512kB (ME) 1*1024kB (E) 2*2048kB (ME) 8*4096k
B (M) = 39960kB
[14173.576253] Node 0 Normal: 116*4kB (UME) 154*8kB (UME) 79*16kB (UME) 89*32kB (UME) 15*64kB (UME) 6*128kB (UE) 4*256kB (UE) 2*512kB (ME) 1*1024kB (M) 2*2048kB
 (UM) 5*4096kB (UM) = 35184kB
[14173.580721] Node 0 hugepages_total=0 hugepages_free=0 hugepages_surp=0 hugepages_size=2048kB
[14173.583030] 3873 total pagecache pages
[14173.585370] 3844 pages in swap cache
[14173.587674] Swap cache stats: add 985621, delete 981777, find 1760/4907
[14173.590036] Free swap  = 0kB
[14173.592396] Total swap = 839676kB
[14173.594747] 1048444 pages RAM
[14173.597093] 0 pages HighMem/MovableOnly
[14173.599437] 81171 pages reserved
[14173.601790] 0 pages hwpoisoned
[14173.604133] [ pid ]   uid  tgid total_vm      rss pgtables_bytes swapents oom_score_adj name
[14173.606579] [  667]     0   667    23500        4   278528      265         0 systemd-journal
[14173.609035] [  694]     0   694    27341        0   221184      851     -1000 systemd-udevd
[14173.611494] [  803]    32   803    16780        0   172032      190         0 rpcbind
[14173.613962] [  806]     0   806    18908        0   151552      172     -1000 auditd
[14173.616444] [  835]    70   835    20688        0   176128      117         0 avahi-daemon
[14173.618947] [  836]     0   836   105552        0   425984      500         0 sssd
[14173.621468] [  838]     0   838     6612        3   102400      303         0 smartd
[14173.624034] [  839]    81   839    22289        0   159744      216      -900 dbus-daemon
[14173.626568] [  841]   997   841     4718        0    69632       42         0 lsmd
[14173.629078] [  843]     0   843   115713        0   405504      979         0 ModemManager
[14173.631586] [  844]     0   844     4223        2    73728       33         0 mcelog
[14173.634125] [  847]   998   847   406490        0   339968     1766         0 polkitd
[14173.636671] [  851]   991   851    21624        0   208896      194         0 rngd
[14173.639209] [  852]     0   852    77086        0   184320      172         0 gssproxy
[14173.641750] [  853]    70   853    20655        0   159744      107         0 avahi-daemon
[14173.644266] [  868]     0   868   107202        0   434176      659         0 sssd_be
[14173.646771] [  869]     0   869   107745        5   471040      402         0 sssd_nss
[14173.649271] [  872]     0   872   124680        2   471040     5485         0 firewalld
[14173.651764] [  876]     0   876    24456        0   221184      683         0 systemd-logind
[14173.654259] [  888]     0   888   150546        0   380928      631         0 NetworkManager
[14173.656755] [  900]     0   900   158766      131   438272     3575         0 tuned
[14173.659244] [  903]     0   903    23242        2   196608      235     -1000 sshd
[14173.661657] [ 1039]     0  1039    10656        0   110592       52         0 atd
[14173.664015] [ 1047]     0  1047    61464        0   110592      229         0 crond
[14173.666320] [ 1054]     0  1054    30419        0   274432      229         0 login
[14173.668572] [ 1401]     0  1401    23435        1   212992      359         0 systemd
[14173.670746] [ 1407]     0  1407    58415        0   299008     1305         0 (sd-pam)
[14173.672848] [ 1413]     0  1413    59214       16    90112      567         0 bash
[14173.674877] [ 1783]     0  1783    20438        1   176128      495         0 dhclient
[14173.676840] [ 2124]     0  2124    54261        0    61440       22         0 sleep
[14173.678729] [ 2138]     0  2138    54261        0    73728       21         0 sleep
[14173.680546] [ 2635]     0  2635  1097137   910253  8818688   186625         0 top_troubleshoo
[14173.682318] [ 2636]     0  2636    68572      103   155648       99         0 top
[14173.684110] Out of memory: Killed process 2635 (top_troubleshoo) total-vm:4388548kB, anon-rss:3641012kB, file-rss:0kB, shmem-rss:0kB, UID:0
```

Заметим на 1-ом скрине, что у нас **Segmentation fault**, а потом через *top* видим, что постоянно увеличивается память и программа падает с ошибкой **Out of memory**, то это, скорее всего, утечка памяти.

9. Для всех процессов, зарегистрированных в системе, определите среднее время непрерывного использования CPU (`CPU burst`) и сформируйте файл со строками вида: `ProcessID=PID : Parent_ProcessID=PPID : Average_Running_Time=ART`. Значение `PPid` берите из `/proc/<PID>/status`. Значение `ART` вычисляйте, разделив `sum_exec_runtime` на `nr_switches`, используя данные из `/proc/<PID>/sched`. Отсортируйте строки по `PPID`.

```bash
#!/bin/bash

> result.txt

for pid in /proc/[0-9]*; do
        PID=${pid#/proc/}

        if [[ -r $pid/status && -r $pid/sched ]]; then
                PPid=$(grep "^PPid:" "$pid/status" | awk '{print $2}')
                sum=$(grep "sum_exec_runtime" "$pid/sched" | awk '{print $3}')
                nr=$(grep "nr_switches" "$pid/sched" | awk '{print $3}')

                if [[ -n "$sum" && -n "$nr" && "$nr" -ne 0 ]]; then
                        ART=$(awk "BEGIN {print $sum / $nr}")
                        echo "ProcessID=$PID : Parent_ProcessID=$PPid : Average_Running_Time=$ART" >> result.txt
                fi
        fi
done

sort -t'=' -k3 -n result.txt -o result.txt
```

```
[root@localhost lab2]# cat result.txt
ProcessID=1 : Parent_ProcessID=0 : Average_Running_Time=1.02395
ProcessID=2 : Parent_ProcessID=0 : Average_Running_Time=0.220008
ProcessID=1039 : Parent_ProcessID=1 : Average_Running_Time=0.849333
ProcessID=1047 : Parent_ProcessID=1 : Average_Running_Time=1.30114
ProcessID=1054 : Parent_ProcessID=1 : Average_Running_Time=2.28388
ProcessID=1401 : Parent_ProcessID=1 : Average_Running_Time=2.48791
ProcessID=1783 : Parent_ProcessID=1 : Average_Running_Time=1.44239
ProcessID=2124 : Parent_ProcessID=1 : Average_Running_Time=5.77699
ProcessID=2138 : Parent_ProcessID=1 : Average_Running_Time=7.193
ProcessID=2707 : Parent_ProcessID=1 : Average_Running_Time=1.53807
ProcessID=667 : Parent_ProcessID=1 : Average_Running_Time=1.09769
ProcessID=694 : Parent_ProcessID=1 : Average_Running_Time=1.304
ProcessID=803 : Parent_ProcessID=1 : Average_Running_Time=0.708464
ProcessID=806 : Parent_ProcessID=1 : Average_Running_Time=0.67931
ProcessID=835 : Parent_ProcessID=1 : Average_Running_Time=0.608423
ProcessID=836 : Parent_ProcessID=1 : Average_Running_Time=0.746986
ProcessID=838 : Parent_ProcessID=1 : Average_Running_Time=0.990698
ProcessID=839 : Parent_ProcessID=1 : Average_Running_Time=0.637606
ProcessID=841 : Parent_ProcessID=1 : Average_Running_Time=0.731628
ProcessID=843 : Parent_ProcessID=1 : Average_Running_Time=0.773843
ProcessID=844 : Parent_ProcessID=1 : Average_Running_Time=0.706597
ProcessID=847 : Parent_ProcessID=1 : Average_Running_Time=0.712611
ProcessID=851 : Parent_ProcessID=1 : Average_Running_Time=0.101067
ProcessID=852 : Parent_ProcessID=1 : Average_Running_Time=0.707822
ProcessID=872 : Parent_ProcessID=1 : Average_Running_Time=1.25694
ProcessID=876 : Parent_ProcessID=1 : Average_Running_Time=1.97184
ProcessID=888 : Parent_ProcessID=1 : Average_Running_Time=0.822953
ProcessID=900 : Parent_ProcessID=1 : Average_Running_Time=1.03444
ProcessID=903 : Parent_ProcessID=1 : Average_Running_Time=1.06661
ProcessID=10 : Parent_ProcessID=2 : Average_Running_Time=0.0526622
ProcessID=11 : Parent_ProcessID=2 : Average_Running_Time=0.112066
ProcessID=12 : Parent_ProcessID=2 : Average_Running_Time=0.245248
ProcessID=13 : Parent_ProcessID=2 : Average_Running_Time=0.208955
ProcessID=144 : Parent_ProcessID=2 : Average_Running_Time=0.01453
ProcessID=145 : Parent_ProcessID=2 : Average_Running_Time=0.024551
ProcessID=146 : Parent_ProcessID=2 : Average_Running_Time=0.189948
ProcessID=147 : Parent_ProcessID=2 : Average_Running_Time=0.0487677
ProcessID=148 : Parent_ProcessID=2 : Average_Running_Time=0.0273903
ProcessID=149 : Parent_ProcessID=2 : Average_Running_Time=0.0135285
ProcessID=150 : Parent_ProcessID=2 : Average_Running_Time=0.0080165
ProcessID=15 : Parent_ProcessID=2 : Average_Running_Time=0.0891299
ProcessID=16 : Parent_ProcessID=2 : Average_Running_Time=0.009019
ProcessID=17 : Parent_ProcessID=2 : Average_Running_Time=0.0842796
ProcessID=18 : Parent_ProcessID=2 : Average_Running_Time=0.520629
ProcessID=19 : Parent_ProcessID=2 : Average_Running_Time=10.8869
ProcessID=20 : Parent_ProcessID=2 : Average_Running_Time=0.0090185
ProcessID=21 : Parent_ProcessID=2 : Average_Running_Time=0.0370765
ProcessID=22 : Parent_ProcessID=2 : Average_Running_Time=0.054613
ProcessID=23 : Parent_ProcessID=2 : Average_Running_Time=0.551209
```

10. В файле из предыдущего пункта после каждой группы записей с одинаковым `PPID` вставьте строку вида: `Average_Running_Children_of_ParentID=N is M`, где `N` — `PPID`, а `M` — среднее значение `ART` по всем процессам данного родителя.

```
#!/bin/bash

awk -F'[ :=]+' '
{
        ppid=$4
        art=$6

        if (NR==1) {
                cur_ppid=ppid
        }

        if (ppid!=cur_ppid) {
                print "Average_Running_Children_of_ParentID=" cur_ppid " is " sum/count
                sum=0
                count=0
                cur_ppid=ppid
        }

        print $0
        sum+=art
        count++
}
END {
        if (count > 0)
                print "Average_Running_Children_of_ParentID=" cur_ppid " is " sum/count
}
' result.txt > result_2.txt
```

```
[root@localhost lab2]# cat result_2.txt
ProcessID=1 : Parent_ProcessID=0 : Average_Running_Time=1.02395
ProcessID=2 : Parent_ProcessID=0 : Average_Running_Time=0.220008
Average_Running_Children_of_ParentID=0 is 0.621979
ProcessID=1039 : Parent_ProcessID=1 : Average_Running_Time=0.849333
ProcessID=1047 : Parent_ProcessID=1 : Average_Running_Time=1.30114
ProcessID=1054 : Parent_ProcessID=1 : Average_Running_Time=2.28388
ProcessID=1401 : Parent_ProcessID=1 : Average_Running_Time=2.48791
ProcessID=1783 : Parent_ProcessID=1 : Average_Running_Time=1.44239
ProcessID=2124 : Parent_ProcessID=1 : Average_Running_Time=5.77699
ProcessID=2138 : Parent_ProcessID=1 : Average_Running_Time=7.193
ProcessID=2707 : Parent_ProcessID=1 : Average_Running_Time=1.53807
ProcessID=667 : Parent_ProcessID=1 : Average_Running_Time=1.09769
ProcessID=694 : Parent_ProcessID=1 : Average_Running_Time=1.304
ProcessID=803 : Parent_ProcessID=1 : Average_Running_Time=0.708464
ProcessID=806 : Parent_ProcessID=1 : Average_Running_Time=0.67931
ProcessID=835 : Parent_ProcessID=1 : Average_Running_Time=0.608423
ProcessID=836 : Parent_ProcessID=1 : Average_Running_Time=0.746986
ProcessID=838 : Parent_ProcessID=1 : Average_Running_Time=0.990698
ProcessID=839 : Parent_ProcessID=1 : Average_Running_Time=0.637606
ProcessID=841 : Parent_ProcessID=1 : Average_Running_Time=0.731628
ProcessID=843 : Parent_ProcessID=1 : Average_Running_Time=0.773843
ProcessID=844 : Parent_ProcessID=1 : Average_Running_Time=0.706597
ProcessID=847 : Parent_ProcessID=1 : Average_Running_Time=0.712611
ProcessID=851 : Parent_ProcessID=1 : Average_Running_Time=0.101067
ProcessID=852 : Parent_ProcessID=1 : Average_Running_Time=0.707822
ProcessID=872 : Parent_ProcessID=1 : Average_Running_Time=1.25694
ProcessID=876 : Parent_ProcessID=1 : Average_Running_Time=1.97184
ProcessID=888 : Parent_ProcessID=1 : Average_Running_Time=0.822953
ProcessID=900 : Parent_ProcessID=1 : Average_Running_Time=1.03444
ProcessID=903 : Parent_ProcessID=1 : Average_Running_Time=1.06661
Average_Running_Children_of_ParentID=1 is 1.46416
ProcessID=10 : Parent_ProcessID=2 : Average_Running_Time=0.0526622
ProcessID=11 : Parent_ProcessID=2 : Average_Running_Time=0.112066
ProcessID=12 : Parent_ProcessID=2 : Average_Running_Time=0.245248
ProcessID=13 : Parent_ProcessID=2 : Average_Running_Time=0.208955
ProcessID=144 : Parent_ProcessID=2 : Average_Running_Time=0.01453
ProcessID=145 : Parent_ProcessID=2 : Average_Running_Time=0.024551
ProcessID=146 : Parent_ProcessID=2 : Average_Running_Time=0.189948
ProcessID=147 : Parent_ProcessID=2 : Average_Running_Time=0.0487677
ProcessID=148 : Parent_ProcessID=2 : Average_Running_Time=0.0273903
ProcessID=149 : Parent_ProcessID=2 : Average_Running_Time=0.0135285
ProcessID=150 : Parent_ProcessID=2 : Average_Running_Time=0.0080165
ProcessID=15 : Parent_ProcessID=2 : Average_Running_Time=0.0891299
ProcessID=16 : Parent_ProcessID=2 : Average_Running_Time=0.009019
ProcessID=17 : Parent_ProcessID=2 : Average_Running_Time=0.0842796
ProcessID=18 : Parent_ProcessID=2 : Average_Running_Time=0.520629
ProcessID=19 : Parent_ProcessID=2 : Average_Running_Time=10.8869
ProcessID=20 : Parent_ProcessID=2 : Average_Running_Time=0.0090185
ProcessID=21 : Parent_ProcessID=2 : Average_Running_Time=0.0370765
```

```
ProcessID=463 : Parent_ProcessID=2 : Average_Running_Time=0.129727
ProcessID=465 : Parent_ProcessID=2 : Average_Running_Time=0.0136667
ProcessID=466 : Parent_ProcessID=2 : Average_Running_Time=0.0279048
ProcessID=467 : Parent_ProcessID=2 : Average_Running_Time=0.0115
ProcessID=479 : Parent_ProcessID=2 : Average_Running_Time=0.0604064
ProcessID=4 : Parent_ProcessID=2 : Average_Running_Time=0.0075155
ProcessID=5034 : Parent_ProcessID=2 : Average_Running_Time=1.56301
ProcessID=5101 : Parent_ProcessID=2 : Average_Running_Time=1.35656
ProcessID=533 : Parent_ProcessID=2 : Average_Running_Time=0.0345
ProcessID=53 : Parent_ProcessID=2 : Average_Running_Time=1.41536
ProcessID=542 : Parent_ProcessID=2 : Average_Running_Time=0.017
ProcessID=576 : Parent_ProcessID=2 : Average_Running_Time=0.763517
ProcessID=577 : Parent_ProcessID=2 : Average_Running_Time=0.019
ProcessID=6 : Parent_ProcessID=2 : Average_Running_Time=0.153874
ProcessID=728 : Parent_ProcessID=2 : Average_Running_Time=0.021
ProcessID=749 : Parent_ProcessID=2 : Average_Running_Time=0.013
ProcessID=767 : Parent_ProcessID=2 : Average_Running_Time=0.283901
ProcessID=768 : Parent_ProcessID=2 : Average_Running_Time=0.0223333
ProcessID=772 : Parent_ProcessID=2 : Average_Running_Time=0.0085
ProcessID=773 : Parent_ProcessID=2 : Average_Running_Time=0.0065
ProcessID=774 : Parent_ProcessID=2 : Average_Running_Time=0.0085
ProcessID=775 : Parent_ProcessID=2 : Average_Running_Time=0.008
ProcessID=776 : Parent_ProcessID=2 : Average_Running_Time=0.015
ProcessID=777 : Parent_ProcessID=2 : Average_Running_Time=0.007
ProcessID=779 : Parent_ProcessID=2 : Average_Running_Time=0.0105
ProcessID=780 : Parent_ProcessID=2 : Average_Running_Time=0.00966667
ProcessID=781 : Parent_ProcessID=2 : Average_Running_Time=0.227721
ProcessID=7 : Parent_ProcessID=2 : Average_Running_Time=0.202548
ProcessID=810 : Parent_ProcessID=2 : Average_Running_Time=0.012
ProcessID=811 : Parent_ProcessID=2 : Average_Running_Time=0.0065
ProcessID=813 : Parent_ProcessID=2 : Average_Running_Time=0.014
ProcessID=8 : Parent_ProcessID=2 : Average_Running_Time=0.0085175
ProcessID=9 : Parent_ProcessID=2 : Average_Running_Time=0.0785687
Average_Running_Children_of_ParentID=2 is 0.314145
ProcessID=853 : Parent_ProcessID=835 : Average_Running_Time=0.448751
Average_Running_Children_of_ParentID=835 is 0.448751
ProcessID=868 : Parent_ProcessID=836 : Average_Running_Time=0.818196
ProcessID=869 : Parent_ProcessID=836 : Average_Running_Time=0.851544
Average_Running_Children_of_ParentID=836 is 0.83487
ProcessID=1413 : Parent_ProcessID=1054 : Average_Running_Time=2.92416
Average_Running_Children_of_ParentID=1054 is 2.92416
ProcessID=1407 : Parent_ProcessID=1401 : Average_Running_Time=0.981001
Average_Running_Children_of_ParentID=1401 is 0.981001
ProcessID=2706 : Parent_ProcessID=1413 : Average_Running_Time=2.00095
Average_Running_Children_of_ParentID=1413 is 2.00095
ProcessID=2714 : Parent_ProcessID=2707 : Average_Running_Time=3.54264
Average_Running_Children_of_ParentID=2707 is 3.54264
ProcessID=6217 : Parent_ProcessID=2714 : Average_Running_Time=0.967069
Average_Running_Children_of_ParentID=2714 is 0.967069
[root@localhost lab2]#
```

11. Используя `/proc`, найдите процесс, которому выделено наибольшее количество оперативной памяти. Сравните результат с выводом `top`.

```bash
#!/bin/bash

max_mem=0
max_pid=0

for pid in /proc/[0-9]*; do
        PID=${pid#/proc/}

        if [[ -r "$pid/status" ]]; then
                mem=$(grep "^VmRSS:" "$pid/status" | awk '{print $2}')

                if [[ -n "$mem" && "$mem" -ge "$max_mem" ]]; then
                        max_mem=$mem
                        max_pid=$PID
                fi
        fi
done

echo "From /proc: "
echo "${max_pid} ${max_mem}"


echo "From top: "
ps -Ao pid=,rss= --sort rss | tail -n 1
```

```
[root@localhost lab2]# ./task11.sh
From /proc:
1 6432
From top:
      1  6432
```

12. Напишите скрипт, изменяющий приоритет процессов, время выполнения которых превышает `N` секунд (`N` задаётся параметром). Для изменения приоритета используйте `renice`. Результат продемонстрируйте с помощью одной из утилит мониторинга, рассмотренных выше.

```bash
#!/bin/bash

N=$1

for pid in $(ps -Ao pid=); do
        time=$(ps -p "$pid" -o etimes=)

        if [[ -n "$time" && "$time" -gt "$N" ]]; then
                renice +5 -p "$pid"
        fi
done
```

```
[root@localhost lab2]# ./task12.sh 5
1 (process ID) old priority 15, new priority 5
2 (process ID) old priority 15, new priority 5
3 (process ID) old priority 15, new priority 5
4 (process ID) old priority 15, new priority 5
6 (process ID) old priority 15, new priority 5
7 (process ID) old priority 15, new priority 5
8 (process ID) old priority 15, new priority 5
9 (process ID) old priority 15, new priority 5
10 (process ID) old priority 15, new priority 5
11 (process ID) old priority 15, new priority 5
12 (process ID) old priority 15, new priority 5
13 (process ID) old priority 15, new priority 5
15 (process ID) old priority 15, new priority 5
16 (process ID) old priority 15, new priority 5
17 (process ID) old priority 15, new priority 5
18 (process ID) old priority 15, new priority 5
19 (process ID) old priority 15, new priority 5
20 (process ID) old priority 15, new priority 5
21 (process ID) old priority 15, new priority 5
22 (process ID) old priority 15, new priority 5
23 (process ID) old priority 15, new priority 5
24 (process ID) old priority 15, new priority 5
25 (process ID) old priority 15, new priority 5
26 (process ID) old priority 15, new priority 5
27 (process ID) old priority 15, new priority 5
28 (process ID) old priority 15, new priority 5
29 (process ID) old priority 15, new priority 5
30 (process ID) old priority 15, new priority 5
53 (process ID) old priority 15, new priority 5
144 (process ID) old priority 15, new priority 5
145 (process ID) old priority 15, new priority 5
146 (process ID) old priority 15, new priority 5
147 (process ID) old priority 15, new priority 5
148 (process ID) old priority 15, new priority 5
149 (process ID) old priority 15, new priority 5
150 (process ID) old priority 15, new priority 5
454 (process ID) old priority 15, new priority 5
455 (process ID) old priority 15, new priority 5
456 (process ID) old priority 15, new priority 5
457 (process ID) old priority 15, new priority 5
458 (process ID) old priority 15, new priority 5
459 (process ID) old priority 15, new priority 5
460 (process ID) old priority 15, new priority 5
461 (process ID) old priority 15, new priority 5
462 (process ID) old priority 15, new priority 5
463 (process ID) old priority 15, new priority 5
465 (process ID) old priority 15, new priority 5
466 (process ID) old priority 15, new priority 5
467 (process ID) old priority 15, new priority 5
```